

問題4-4: EDFA 前置増幅構成での QPSK の BER vs 入力 パワー P_{in}

問題

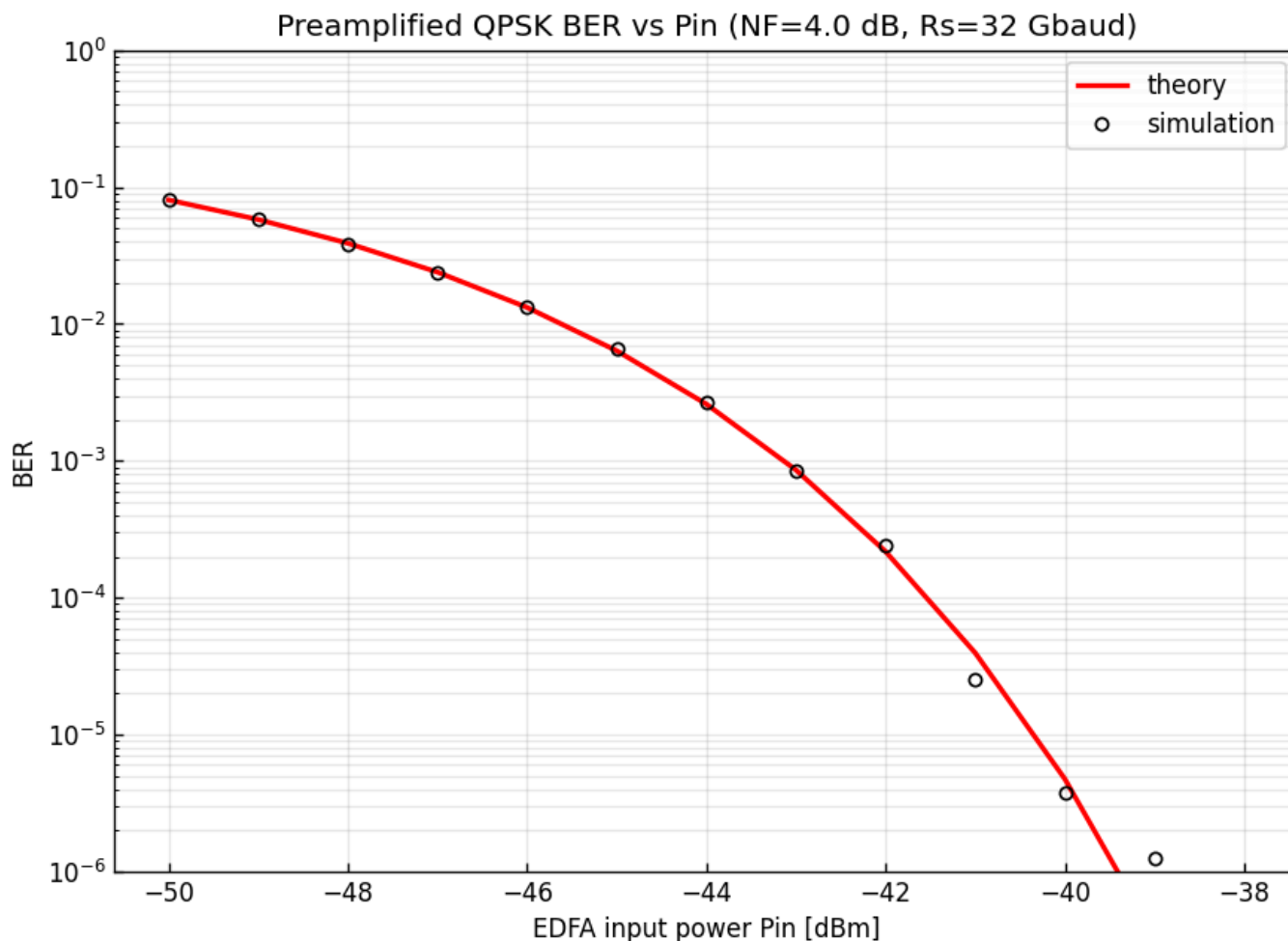
単一偏波 QPSK 光信号を EDFA で増幅した後、コヒーレント光受信する(前置増幅構成)。EDFA から生じる ASE 雑音は複素 AWGN でモデル化でき、雑音指数 $NF = 4\text{ dB}$ とする。このときの BER の EDFA 入力パワー P_{in} 依存性を求める(レーザ位相雑音は無視)。

実行結果(出力)

$R_s=32\text{ Gbaud}$, $NF=4.0\text{ dB}$, $G=20.0\text{ dB}$
 $h\nu = 1.282\text{e-}19\text{ J}$, $S_{ASE} = 1.593\text{e-}17\text{ W/Hz}$

$P_{in}[\text{dBm}]$	$E_s/N_0[\text{dB}]$	BER(sim)	BER(theory)
-50	2.92	$8.04\text{e-}02$	$8.07\text{e-}02$
-45	7.92	$6.54\text{e-}03$	$6.38\text{e-}03$
-43	9.92	$8.43\text{e-}04$	$8.59\text{e-}04$
-40	12.92	$3.75\text{e-}06$	$4.75\text{e-}06$
-38	14.92	$0.00\text{e+}00$	$1.24\text{e-}08$

受信感度 (BER= $1\text{e-}3$) $\approx -43.1\text{ dBm}$

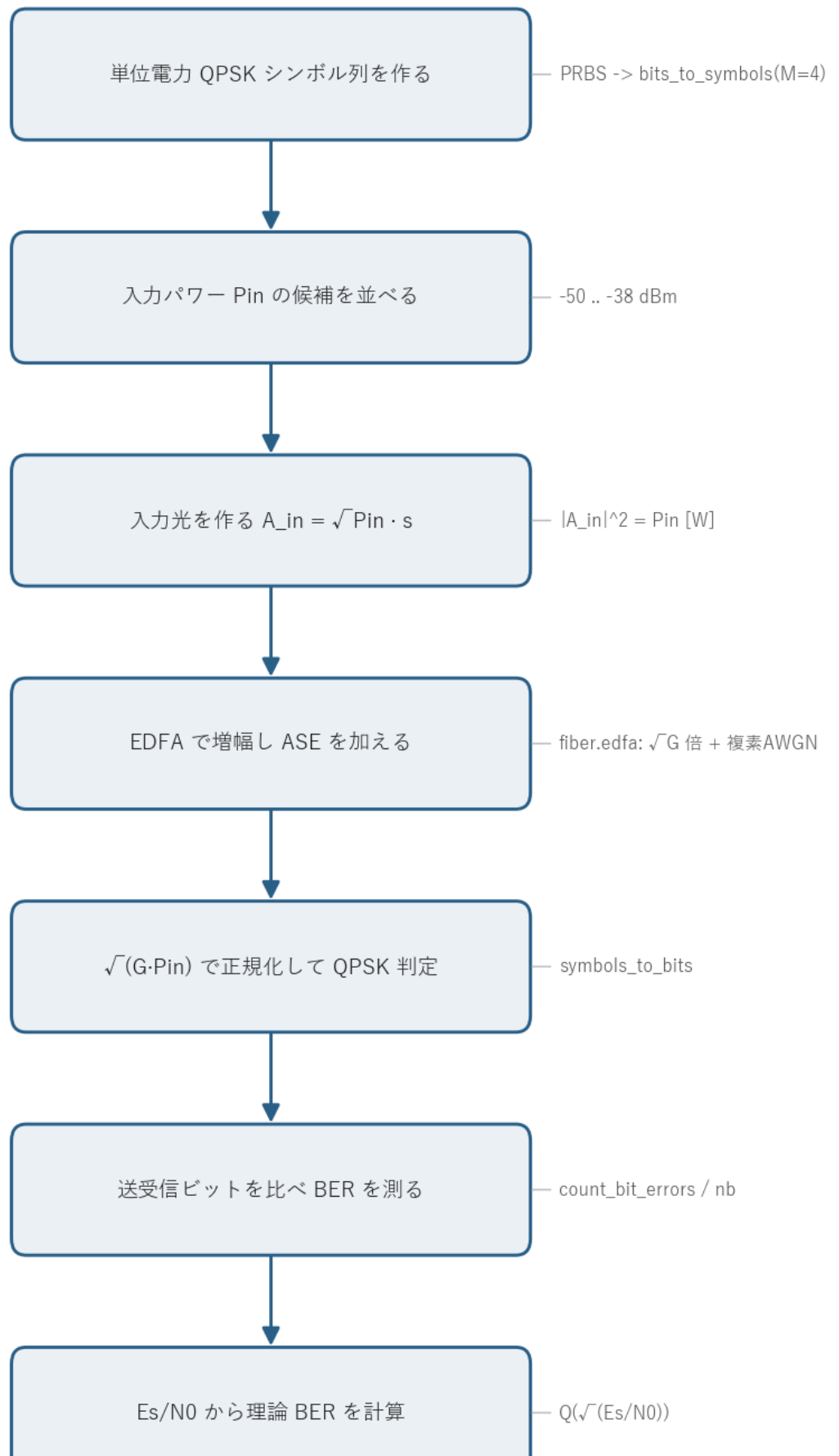


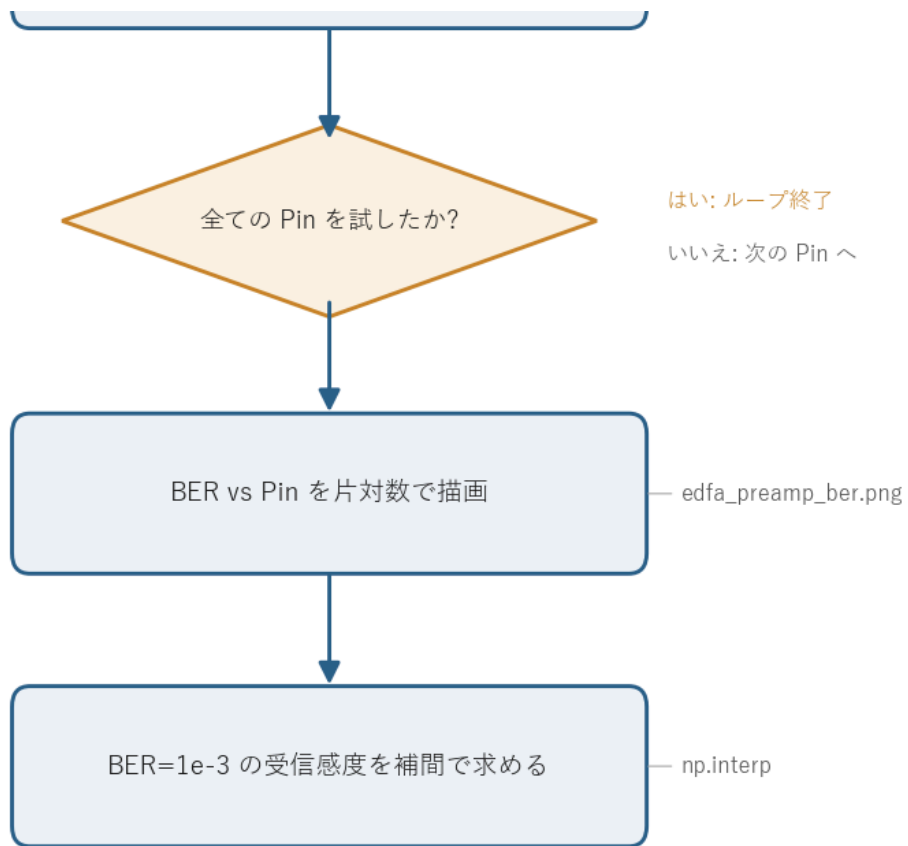
図の読み方 - 横軸が EDFA への入力パワー P_{in} 、縦軸が BER(片対数)。 - 実線が理論値 $Q(\sqrt{E_s/N_0})$ 、○がモンテカルロ・シミュレーション。両者が重なる。 - P_{in} を上げるほど BER が急激に下がり、BER = 10^{-3} を切る入力パワー(受信感度)は約 **-43 dBm**。これは前置増幅コヒーレント受信機の典型値。

0. 処理の流れ

`solution.py` を実行すると、次の順で処理が進む。

EDFA 前置増幅 QPSK の BER vs Pin の処理フロー





最初に単位電力の QPSK シンボル列を 1 本だけ作っておき、その同じ系列を使い回す。あとは入力パワー P_{in} を変えながら「入力光を作る → EDFA に通す → 判定して BER を測る」を繰り返すループになっている。分岐は「全ての P_{in} を試したか」の 1 箇所だけで、ループを抜けたら BER 曲線を描き、受信感度を補間で求めて終わる。シンボル生成と EDFA・判定が分離しているので、雑音の効果だけを P_{in} 掃引で取り出せる。

1. 前置増幅構成と ASE 雑音

微弱な受信光をそのまま検出すると SNR が足りない。そこで受信機の前に **EDFA(エルビウム添加光ファイバ増幅器)** を置いて増幅する(**前置増幅, preamplifier**)。ただし EDFA は信号を増幅すると同時に、自然放出光由来の **ASE 雑音(Amplified Spontaneous Emission, 増幅された自然放出光)** を加える。前置増幅構成では、これが 主要な雑音源になる。

ASE 雑音は広帯域で各周波数成分が独立、振幅はガウス分布に従うので **複素 AWGN(加法的白色ガウス雑音)** でモデル化できる。EDFA 出力での ASE のパワースペクトル密度(1 偏波・片側)は

$$S_{ASE} = n_{sp} h\nu (G - 1), \quad n_{sp} = \frac{NF_{lin}}{2},$$

で与えられる。各記号の意味は次の通り。

- $h\nu$ は光子 1 個のエネルギー。1550 nm では $h\nu = 1.28 \times 10^{-19}$ J。
- G は EDFA の利得(線形値)。本問では 20 dB なので $G = 100$ 。
- n_{sp} は自然放出係数で、反転分布が完全なら 1、不完全だと 1 より大きくなる。
- NF_{lin} は雑音指数の線形値。高利得近似で $NF_{lin} = 2n_{sp}$ の関係がある。NF=4 dB なら $NF_{lin} = 10^{0.4} \approx 2.51$ 、よって $n_{sp} \approx 1.26$ 。

$(G - 1)$ という因子は、自然放出された 1 個の光子が利得媒質を通して G 倍に増幅され、もとの 1 個を差し引いた 正味の増分を表す。利得が高いほど ASE も増えるが、後で見るように信号も同じ G 倍されるので、SNR は G によらなくなる。

solution.py の出力にある $S_{ASE} = 1.593e-17$ W/Hz は、この式に $G = 100$ 、 $n_{sp} = 1.26$ 、 $h\nu = 1.28 \times 10^{-19}$ を入れた値で、ここを起点に雑音電力を組み立てていく。

2. SNR と P_{in} の関係

ASE は PSD(単位帯域あたりの電力)なので、SNR を出すには受信帯域を決めて電力に直す必要がある。ここでは 1 シンボルを 1 サンプルで表す(1 sample/symbol, $f_s = R_s$)モデルを使う。すると有効帯域は符号速度 R_s ぶんになり、1 シンボルあたりの ASE 雑音電力は $S_{ASE} \cdot R_s$ になる。一方、増幅後の信号電力は $G P_{in}$ 。両者の比が 1 シンボル SNR($= E_s/N_0$)で、

$$\frac{E_s}{N_0} = \frac{G P_{in}}{S_{ASE} R_s} = \frac{G P_{in}}{n_{sp} h\nu (G - 1) R_s} \xrightarrow{G \gg 1} \frac{P_{in}}{n_{sp} h\nu R_s} = \frac{2P_{in}}{NF_{lin} h\nu R_s}.$$

高利得 $G \gg 1$ では $G/(G - 1) \rightarrow 1$ となり、利得 G が式から消える。これは増幅で信号も雑音も同じだけ大きくなるため、SNR は P_{in} に正比例し、利得をいくら上げて改善しない。前置増幅の役割は SNR を稼ぐことではなく、後段の受信機雑音(熱雑音やショット雑音)に対して信号を持ち上げ、ASE 限界まで性能を引き出すことにある。

最終形 $E_s/N_0 = 2P_{in}/(NF_{lin} h\nu R_s)$ を見ると、感度を決めるのは NF_{lin} と $h\nu R_s$ だけ。NF が小さいほど、また符号速度が低いほど、同じ P_{in} で SNR が高くなる。

QPSK は I 軸・Q 軸が独立な 2 つの BPSK と等価なので、BER は

$$BER = Q\left(\sqrt{E_s/N_0}\right)$$

になる。 $Q(x)$ は標準正規分布の上側確率で、SNR が上がると指数関数的に小さくなる。BER 曲線が片対数で ほぼ直線的に急降下するのはこのため。

3. シミュレーションと一致

各 P_{in} について、単位電力 QPSK シンボル s に $\sqrt{P_{\text{in}}}$ を掛けて光パワー P_{in} の入力光 $\sqrt{P_{\text{in}}}s$ を作り、`fiber.edfa` に通す($\sqrt{G}$ 倍 + ASE 付加)。出力を $\sqrt{GP_{\text{in}}}$ で正規化してから QPSK 硬判定し、送信ビットと比べて BER を測る。実行結果のとおり、シミュレーション値は理論値とよく一致する。 $P_{\text{in}} = -38$ dBm で `BER(sim)=0` になっているのは、誤りが出るほど SNR が高くないためで、40 万シンボルでは 1 個も誤らなかったことを意味する(理論値 1.24×10^{-8} なら期待誤り数は **0.01** 個以下)。

受信感度(BER = 10^{-3} を達成する最小 P_{in})は約 **-43** dBm。BER = 10^{-3} には $E_s/N_0 \approx 9.8$ dB が必要で、これを上の SNR 式に代入すると $P_{\text{in}} \approx -43$ dBm が得られる。1 シンボルあたりに換算すると おおよそ十数個の光子で誤りなく受信できる感度で、コヒーレント前置増幅受信機の量子限界に近い高感度性を示す。

位相雑音を無視する意味 本問はレーザ位相雑音を無視するので、劣化要因は ASE だけになる。実機では問3 の位相回復 DSP で 位相雑音を別途補償するため、ここで見た ASE 限界の性能をそのまま議論できる。

4. 関数ごとの詳細

`solution.py` は補助関数 `dbm_to_w` と、全体をつなぐ `main` から成る。物理計算と通信処理は `_common/fiber.py` と `_common/comm.py` の関数に委ねている。

`dbm_to_w(dbm)`

電力の単位 dBm をワットに直す小さな関数。

項目	内容
引数	<code>dbm</code> : 電力 dBm 。配列も可。
戻り値	電力 [W]。

中身は `10 ** (dbm / 10.0) * 1e-3` の 1 行。dBm は 1 mW 基準なので、`10**(dbm/10)` で mW に直し、`1e-3` を掛けて W にする。例えば **-43** dBm なら $10^{-4.3} \times 10^{-3} \approx 5.0 \times 10^{-8}$ W。

`fiber.ase_psd(gain, nf_db)`

EDFA 出力での ASE パワースペクトル密度 S_{ASE} を返す(`_common/fiber.py`)。

項目	内容
引数	<code>gain</code> : 利得 G (線形値)。 <code>nf_db</code> : 雑音指数 [dB]。 <code>wavelength_nm</code> : 波長(既定 1550 nm)。

項目	内容
戻り値	S_{ASE} W/Hz。

処理は次の通り。

1. `nf_lin = 10 ** (nf_db / 10.0)` で NF を線形値に直す。
2. `n_sp = nf_lin / 2.0` で自然放出係数を出す(高利得近似 $\text{NF}_{\text{lin}} = 2n_{\text{sp}}$)。
3. `n_sp * photon_energy() * (gain - 1.0)` で $S_{\text{ASE}} = n_{\text{sp}} h \nu (G - 1)$ を返す。

第 1 節の式そのまま、`main` の冒頭でこの値を表示している。

```
fiber.edfa(A, gain, nf_db, fs, rng)
```

光信号を増幅し、ASE 雑音を付加する EDFA モデル(`_common/fiber.py`)。

項目	内容
引数	<code>A</code> : 入力光振幅(単位 $\sqrt{\text{W}}$ 、 $ A ^2$ が光パワー)。 <code>gain</code> : G 。 <code>nf_db</code> : NF。 <code>fs</code> : サンプルングレート [Hz]。 <code>rng</code> : 乱数生成器。
戻り値	増幅 + ASE 付加後の複素振幅。

内部処理。

1. `s_ase = ase_psd(gain, nf_db)` で PSD を求める。
2. `var = s_ase * fs` で、帯域 `fs` ぶんの複素雑音の総分散にする。PSD に帯域を掛けると電力になる。
3. `sigma = np.sqrt(var / 2.0)` で 1 軸(実部・虚部それぞれ)の標準偏差を出す。複素雑音は実部と虚部に分散を半分ずつ分配するので $\sqrt{\text{var}/2}$ になる。
4. `noise = sigma * (rng.standard_normal(...) + 1j * rng.standard_normal(...))` で複素ガウス雑音を作る。
5. `np.sqrt(gain) * A + noise` を返す。振幅は $\sqrt{G}$ 倍(電力は G 倍)で、そこに ASE を足す。

`var = s_ase * fs` の 1 行が「PSD × 帯域 = 電力」という ASE 雑音電力の本体で、ここで $f_s = R_s$ を渡すから第 2 節の「1 シンボルあたり $S_{\text{ASE}} R_s$ 」と整合する。

```
comm.bits_to_symbols(bits, M) / comm.symbols_to_bits(sym, M)
```

ビット列とシンボル列を相互変換する(`_common/comm.py`)。 `M=4` で QPSK になる。

- `bits_to_symbols` : 2 ビットごとに 1 つの単位電力 QPSK シンボル(複素数)へマップする。グレイ符号化。

- `symbols_to_bits`: 受信シンボルを最近傍判定(硬判定)してからビットに戻す。雑音入りの受信点でも、内部で各軸を一番近い基準点に丸めるので、そのまま BER 計算に使える。

QPSK は判定が I 軸・Q 軸の符号を見るだけなので、振幅スケールに依らない。`main` で $\sqrt{GP_{in}}$ で正規化しているのは判定上は不要だが、受信点の意味を明示するために入れている。

```
comm.ber_theory_qam(M, snr_db)
```

方形 M-QAM の BER 解析解(近似式)を返す。QPSK では厳密解 $Q(\sqrt{E_s/N_0})$ に一致する。`main` では各 P_{in} の E_s/N_0 を dB に直して渡し、理論曲線を描いている。

```
main()
```

全体をつなぐ関数。

1. 乱数生成器を初期化し、 $G = 10^{20/10} = 100$ 、 S_{ASE} 、 $h\nu$ を求めて表示する。
2. PRBS から単位電力 QPSK シンボル列 `s` を作り、40 万シンボルまでタイル状に並べる。送信ビット `bits_tx` も作る。
3. $P_{in} = -50$ から -38 dBm まで 1 dB 刻みで掃引する。各 P_{in} で次をやる。
 - `A_in = sqrt(Pin) * s` で入力光を作る。
 - `A_out = fiber.edfa(A_in, G, NF, RS, rng)` で増幅 + ASE。
 - `r = A_out / sqrt(G*Pin)` で正規化し、`symbols_to_bits` で判定。
 - 送受信ビットを比べ `count_bit_errors / nb` で BER を出す。
 - SNR 式 `snr_lin = G*Pin / (S_ASE*RS)` から理論 BER も計算する。
4. 表を表示し、BER vs P_{in} を片対数で描いて `edfa_preamp_ber.png` に保存する(0 の点是对数軸に載らないので除外)。
5. BER 曲線を補間し、 $BER = 10^{-3}$ を切る受信感度を求めて表示する。

図のラベルは日本語フォントに依存しないよう英語にしている。

5. 実行

```
python solution.py
```

標準出力は冒頭の「実行結果(出力)」、生成図は `edfa_preamp_ber.png` を参照。処理フロー図 `flow.png` は `python make_flow.py` で作り直せる。

6. この問題のつながり

ここでは EDFA 1 台(前置増幅)だけを扱った。次の 問4-5 では、ファイバ損失 + EDFA を 1 スパンとして多中継伝送(1~50 スパン)を行い、スパン数が増えると ASE が累積して BER が劣化する様子を見る。問

4-6 で波長分散、**問4-7** で非線形効果を順に加えていく。